1.Design a simple LAN consisting of two PCs connected through a switch using Cisco Packet Tracer. Configure appropriate IP addresses for both hosts and verify connectivity using the ping command. Simultaneously capture the packet exchange using Wireshark. Identify the Ethernet frame fields such as Source MAC Address, Destination MAC Address, EtherType, and Frame Check Sequence (FCS). Explain how the Data Link layer uses MAC addressing to ensure successful frame delivery within the local network.

1. Aim

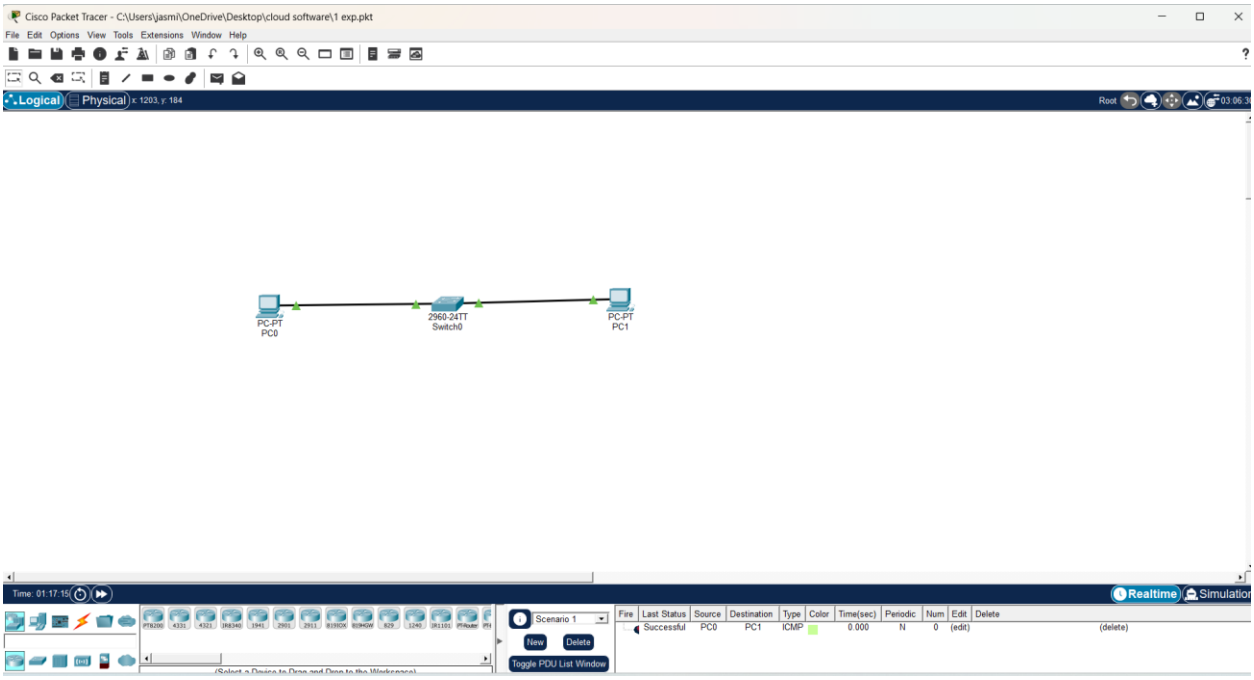
To design a simple Local Area Network (LAN) consisting of two PCs connected through a switch in Cisco Packet Tracer, to configure IP addresses on both hosts, to verify Layer-3 connectivity using the ping command, and to capture and analyse the exchanged Ethernet frames using Wireshark — identifying the Source MAC Address, Destination MAC Address, EtherType and Frame Check Sequence (FCS) fields, and to understand how MAC addressing at the Data Link layer ensures successful frame delivery within a local network.

2. Procedure

1. Open Cisco Packet Tracer and create a new blank workspace; place two Generic PCs (PC0, PC1) and one Cisco 2960 switch.
2. Connect PC0 to Switch0 port Fa0/1 and PC1 to Switch0 port Fa0/2 using Copper Straight-Through cables, and confirm the link-status lights turn green at both ends.
3. Open PC0 → Desktop → IP Configuration and assign a static IP address 192.168.1.1 with subnet mask 255.255.255.0.
4. Open PC1 → Desktop → IP Configuration and assign a static IP address 192.168.1.2 with subnet mask 255.255.255.0.
5. Open the Command Prompt on PC0 and verify connectivity to PC1 using the ping command.
6. Clear PC0's ARP cache and start a Wireshark capture (or Packet Tracer Simulation mode) on PC0's interface.
7. Re-issue the ping and observe the ARP Request/Reply and ICMP Echo Request/Reply frames as they appear in the Wireshark packet list.
8. Click a captured frame and expand its “Ethernet II” section to identify and record the Source MAC Address, Destination MAC Address, EtherType and Frame Check Sequence (FCS).

3. Screenshots and Observations

Figure 1: Topology Built in Packet Tracer

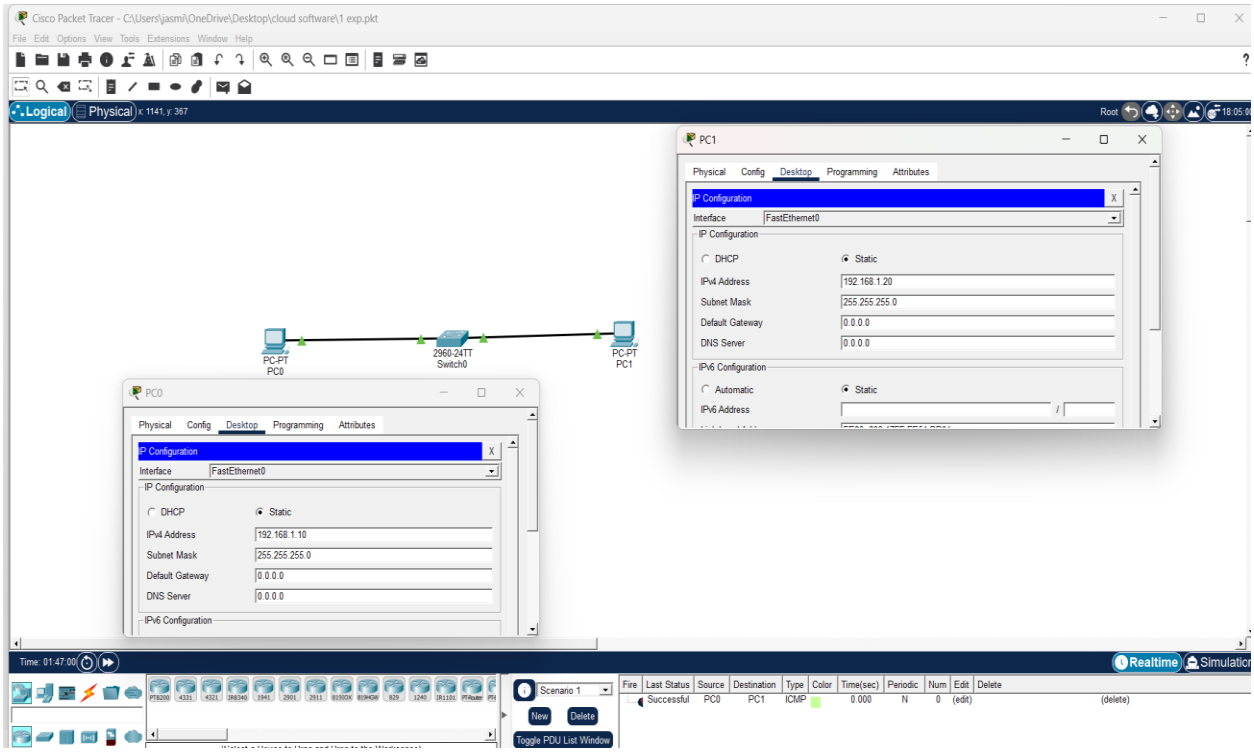


PC0 – Switch0 – PC1 physical topology

Result: PC0, PC1 and Switch0 connected by straight-through cables; link lights green.

Explanation: Green link lights confirm the Physical and Data Link layer connection is up, giving Ethernet frames a hardware path between the two hosts.

Figure 2: IP Address Configuration



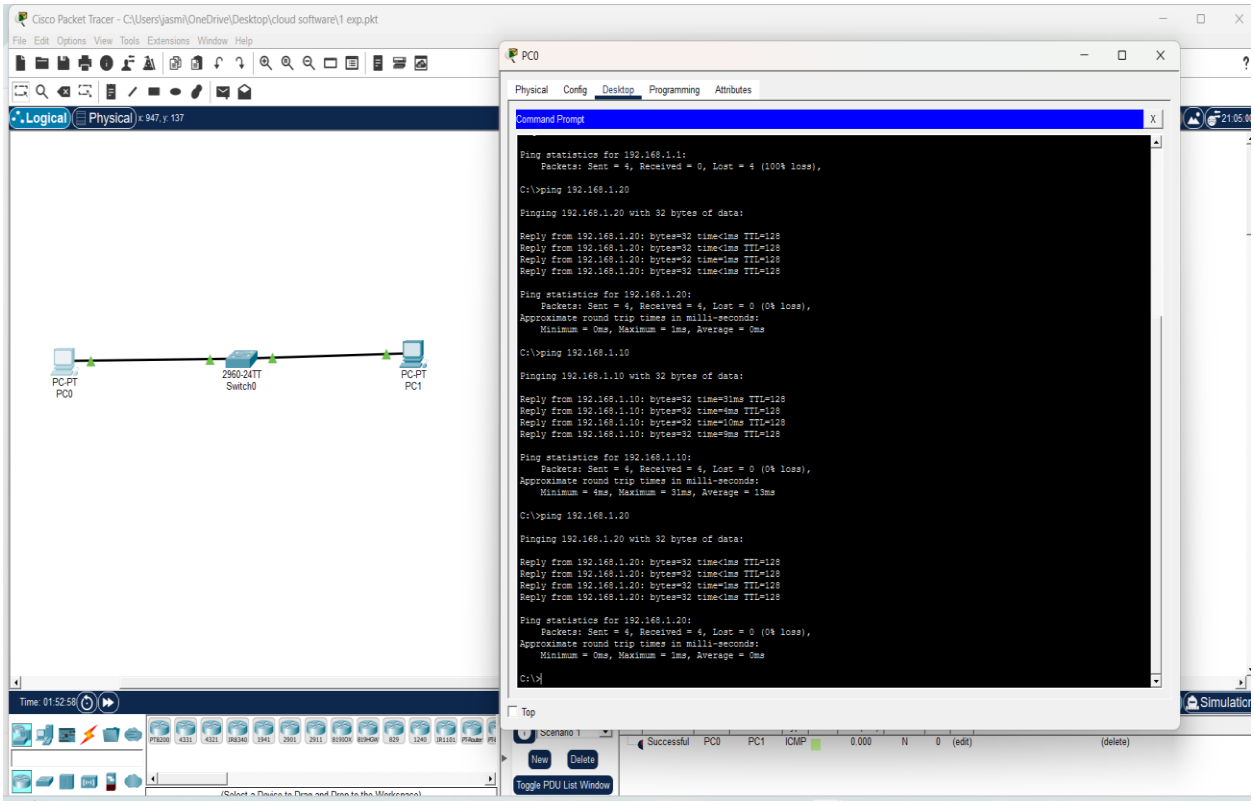
IP Configuration window of PC0 / PC1

Device	IP Address	Subnet Mask	Connected Port
PC0	192.168.1.10	255.255.255.0	Switch0 Fa0/1
PC1	192.168.1.20	255.255.255.0	Switch0 Fa0/2

Result: PC0 = 192.168.1.10, PC1 = 192.168.1.20, both /24, no gateway needed.

Explanation: Both hosts are on the same subnet, so PC0 addresses frames to PC1 directly over the switch instead of via a router.

Figure 3: Ping Verification



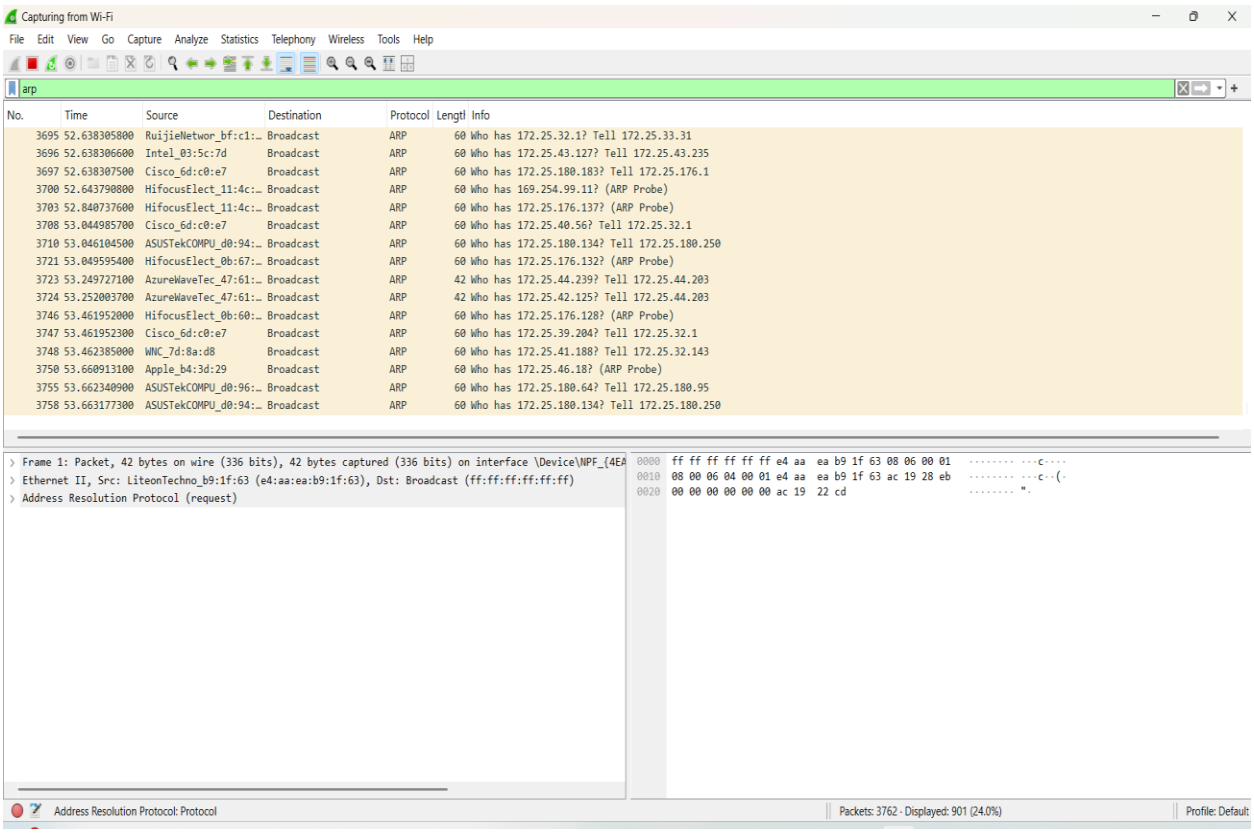
Command Prompt: successful ping from PC0 to PC1

Command: ping 192.168.1.20

Result: Reply from 192.168.1.20, 0% packet loss.

Explanation: Confirms end-to-end IP connectivity; this exchange only completes once the Data Link layer has resolved PC1's MAC address.

Figure 4: Wireshark — ARP Request/Reply Capture



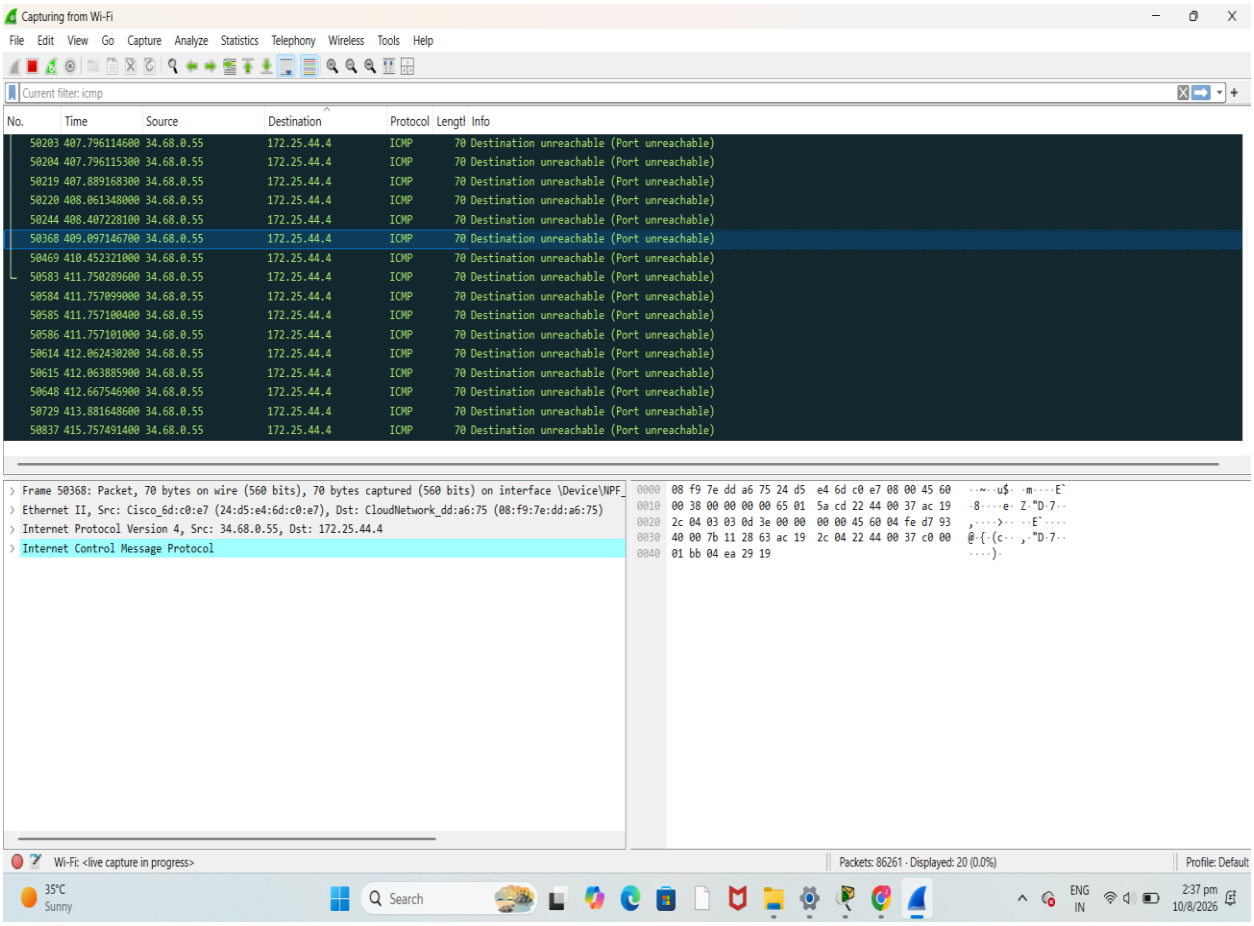
Wireshark packet list showing the ARP exchange

Command: arp -d then ping 192.168.1.20

Result: ARP Request (broadcast, FF:FF:FF:FF:FF:FF) followed by a unicast ARP Reply from PC1.

Explanation: This is MAC address resolution: PC0 broadcasts to the whole LAN, but only PC1 replies, and it replies as a unicast frame.

Figure 5: Wireshark — ICMP Echo Request/Reply Capture



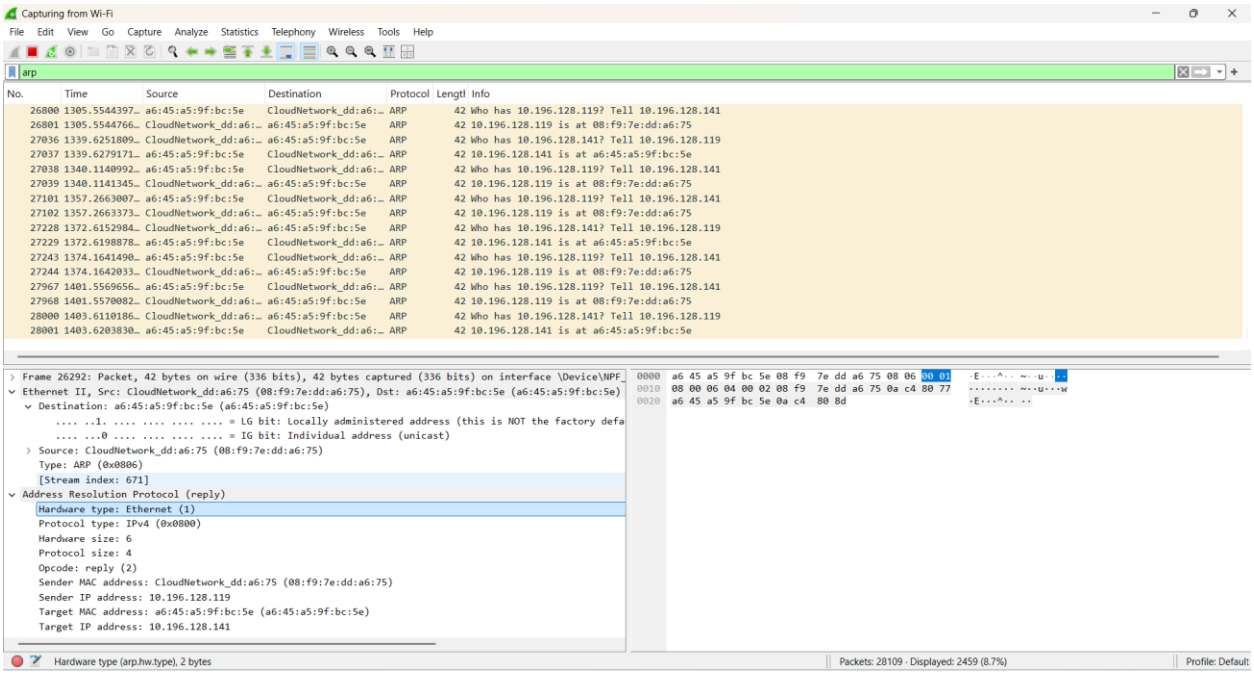
Wireshark packet list filtered to icmp

Command: ping 192.168.1.20

Result: Echo Request and Echo Reply frames exchanged directly between PC0 and PC1's MAC addresses.

Explanation: With the MAC address now known, the switch forwards each frame out a single port instead of flooding it, making delivery both targeted and efficient.

Figure 6: Wireshark — Ethernet II Frame Details



Packet details pane with the Ethernet II section expanded

Result: Destination MAC, Source MAC, EtherType (0x0806 for ARP / 0x0800 for IPv4) and a valid FCS are visible.

Explanation: These fields are what let the network deliver the frame correctly: Destination MAC tells a NIC whether to accept it, Source MAC lets the switch learn where replies go, EtherType tells the host which protocol to hand the payload to, and the FCS lets the NIC confirm the frame was not corrupted before accepting it.

Ethernet Frame Field	ARP Request Frame	ICMP Echo Request Frame
Source MAC Address	MAC of PC0 (sender)	MAC of PC0 (sender)
Destination MAC Address	FF:FF:FF:FF:FF:FF (broadcast)	MAC of PC1 (learned via ARP)
EtherType	0x0806 (ARP)	0x0800 (IPv4)
Frame Check Sequence (FCS)	4-byte CRC (validated by NIC)	4-byte CRC (validated by NIC)
Frame Length	To be filled from capture	To be filled from capture

4. Result and Discussion

The ping command from PC0 to PC1 completed successfully with 0% packet loss, confirming that both hosts were correctly configured on the same IP subnet (192.168.1.0/24) and that the switch correctly forwarded frames between them. The Wireshark capture showed that connectivity at Layer 3 (IP) was made possible only after the Data Link layer resolved the destination's MAC address through ARP: the source host broadcasts an ARP Request (Destination MAC = FF:FF:FF:FF:FF:FF) asking for the owner of 192.168.1.20, and the owning host replies with a unicast ARP Reply containing its MAC address, which is then cached and used to address the following ICMP frames directly.

From the analysis, the Data Link layer ensures successful frame delivery within the local network through the following mechanisms:

- Unique MAC addressing: every NIC has a globally unique 48-bit MAC address, allowing frames to be addressed to one specific device on the LAN.
- Address resolution: ARP maps the destination's IP address to its MAC address before any unicast frame can be constructed, since Ethernet hardware forwards on MAC addresses, not IP addresses.
- Switch learning and forwarding: the switch inspects the Source MAC of incoming frames to build its MAC address table, and uses the Destination MAC of each frame to forward it out only the correct port, avoiding unnecessary flooding once the destination is known.
- Destination filtering at the NIC: every NIC checks the Destination MAC Address of each frame it receives and accepts only frames addressed to itself, its broadcast address, or a joined multicast group, so only the intended host processes the frame.
- Error detection via FCS: the 4-byte CRC in the FCS field lets the receiving NIC verify that the frame arrived unaltered; a checksum mismatch causes the frame to be silently discarded, preventing corrupted data from being passed up the protocol stack.

Together, these mechanisms allow the Data Link layer to deliver frames reliably and efficiently between two directly connected hosts on the same LAN segment, independent of any Layer-3 routing.

5. Conclusion

A simple LAN with two PCs and a switch was successfully designed and configured in Cisco Packet Tracer. IP connectivity between the two hosts was verified using the ping command, and the underlying Ethernet frame exchange was captured and analysed using Wireshark. The Source MAC Address, Destination MAC Address, EtherType and Frame Check Sequence (FCS) fields were identified within the captured frames. The experiment demonstrated that the Data Link layer, through MAC addressing, ARP-based address resolution, switch MAC-table forwarding, and FCS-based error checking, ensures that frames are delivered accurately and efficiently to the correct host within a local network.

3.Construct a local area network with multiple hosts connected through a switch in Cisco Packet Tracer. Clear the ARP cache on one of the hosts and initiate communication with another host using the ping command. Capture the communication in Wireshark and identify the ARP Request and ARP Reply packets. Analyze the sender and target MAC/IP addresses and explain how the Address Resolution Protocol resolves logical IP addresses into physical MAC addresses before data transmission.

Aim

To construct a Local Area Network (LAN) with multiple hosts connected through a switch in Cisco Packet Tracer, clear the ARP cache on one host, initiate a ping to another host, capture the exchange in Wireshark, and analyze the ARP Request/Reply packets to understand how ARP resolves IP addresses to MAC addresses before data transmission.

Procedure

1. Open Cisco Packet Tracer and create a new project.
2. Place one Switch (2960) and at least three PCs (PC0, PC1, PC2) in the workspace.
3. Connect each PC to the switch using Copper Straight-Through cables (PC FastEthernet port → Switch FastEthernet port).
4. Wait until all link indicators turn green (enable Simulation mode → Realtime, or wait for STP convergence).
5. Configure static IP addresses and subnet masks on each PC via Desktop → IP Configuration, ensuring all hosts belong to the same subnet (e.g., 192.168.1.0/24).
6. Verify connectivity is properly configured by checking IP settings on all PCs.
7. Open the Command Prompt on the source PC (e.g., PC0).
8. Clear the ARP cache using the command: arp -d *
9. Verify the ARP cache is empty using: arp -a
10. Switch Packet Tracer to Simulation Mode.
11. From the source PC's command prompt, ping the destination PC (e.g., PC1): ping 192.168.1.2
12. In Simulation Mode, click Capture/Forward step-by-step to observe the packets generated — first an ARP Request (broadcast) followed by an ARP Reply (unicast), and then the ICMP Echo Request/Reply.
13. Click on each ARP packet in the Simulation Panel and open the PDU details to inspect the packet contents (or use Wireshark if bridged via a real NIC).
14. Note the Sender MAC, Sender IP, Target MAC, and Target IP fields in both the ARP Request and ARP Reply.
15. Re-check the ARP cache on the source PC using arp -a after the ping completes to confirm the new entry has been learned.

16. Record and analyze all observations.

Network Topology

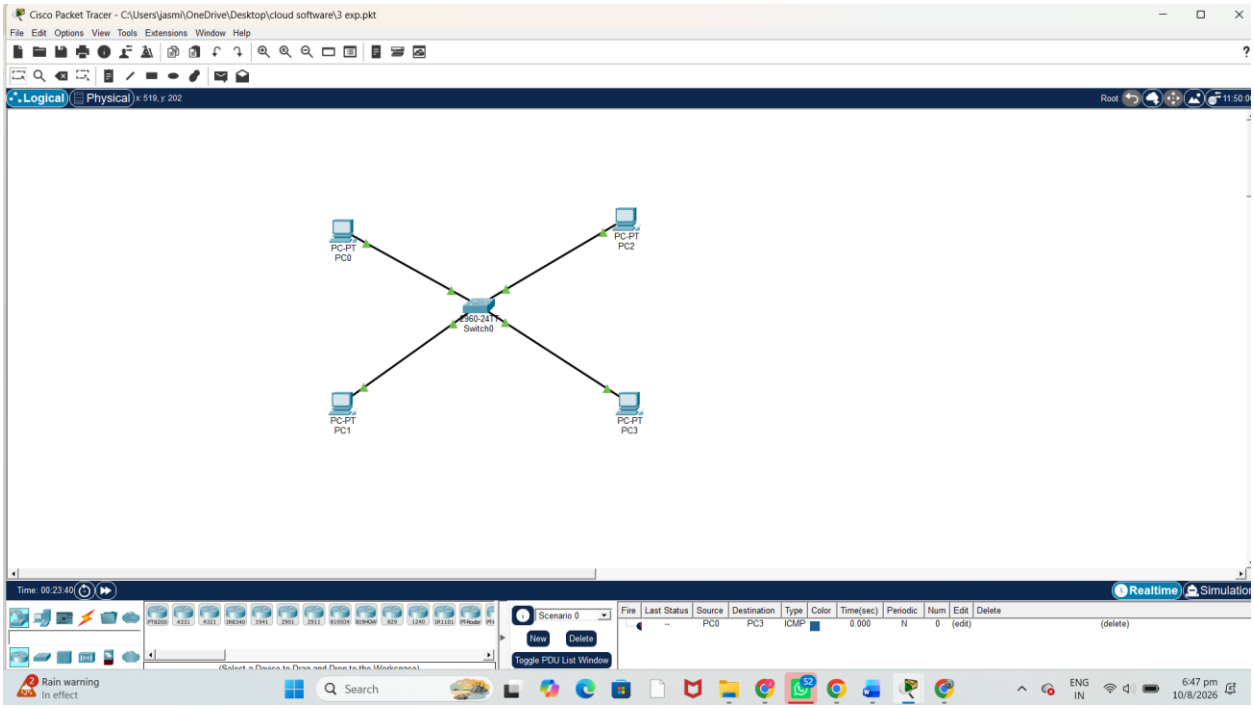


Figure 1: LAN Topology – Switch connected to PC0, PC1, PC2

IP Address Configuration

Device	IP Address	Subnet Mask	Default Gateway
PC0	192.168.1.1	255.255.255.0	Not required (same subnet)
PC1	192.168.1.2	255.255.255.0	Not required (same subnet)
PC2	192.168.1.3	255.255.255.0	Not required (same subnet)

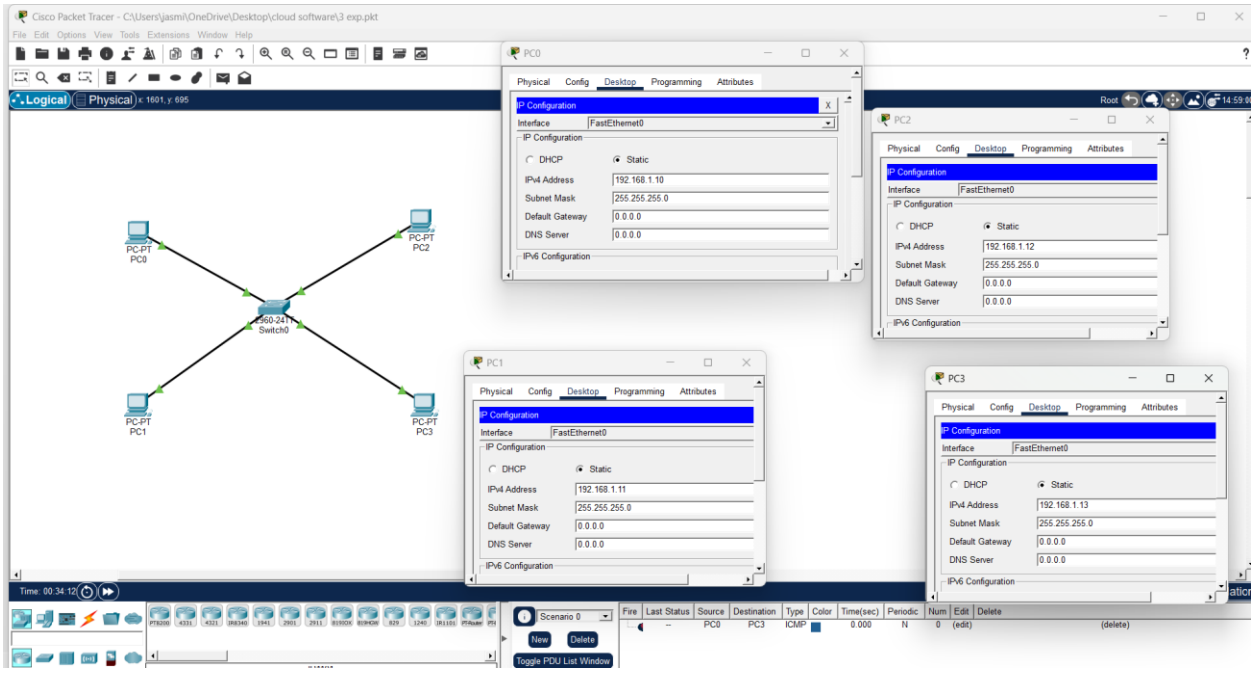


Figure 2: IP Configuration window of PC0

Wireshark / Simulation Capture

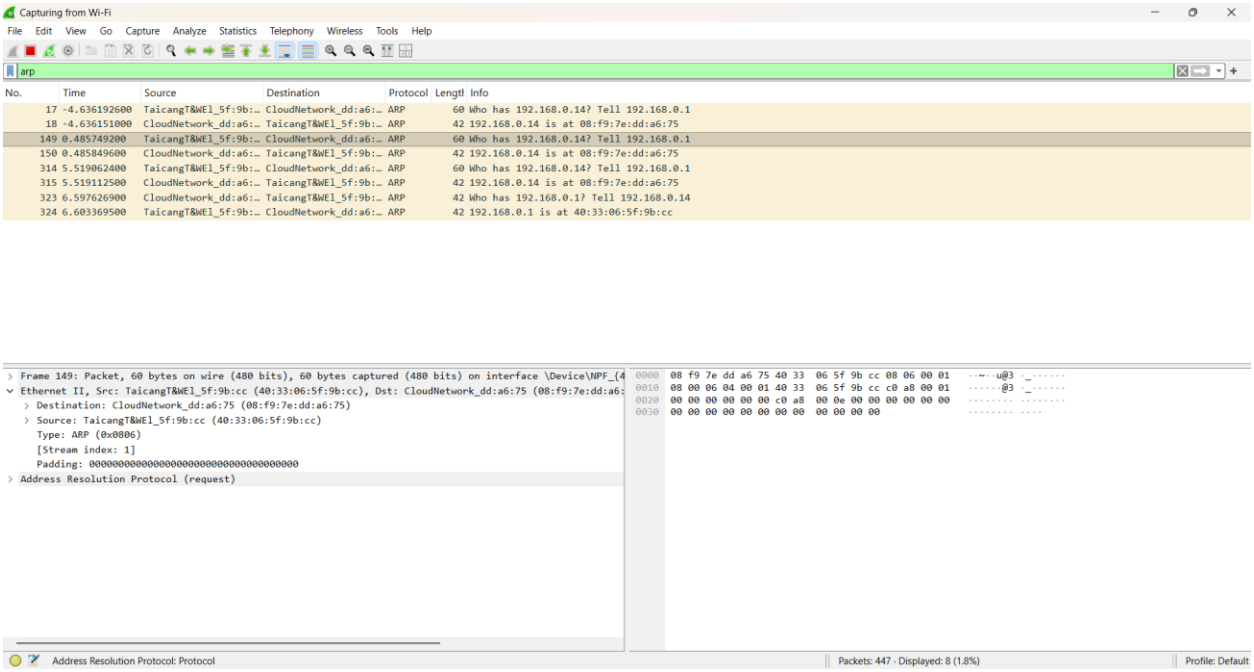
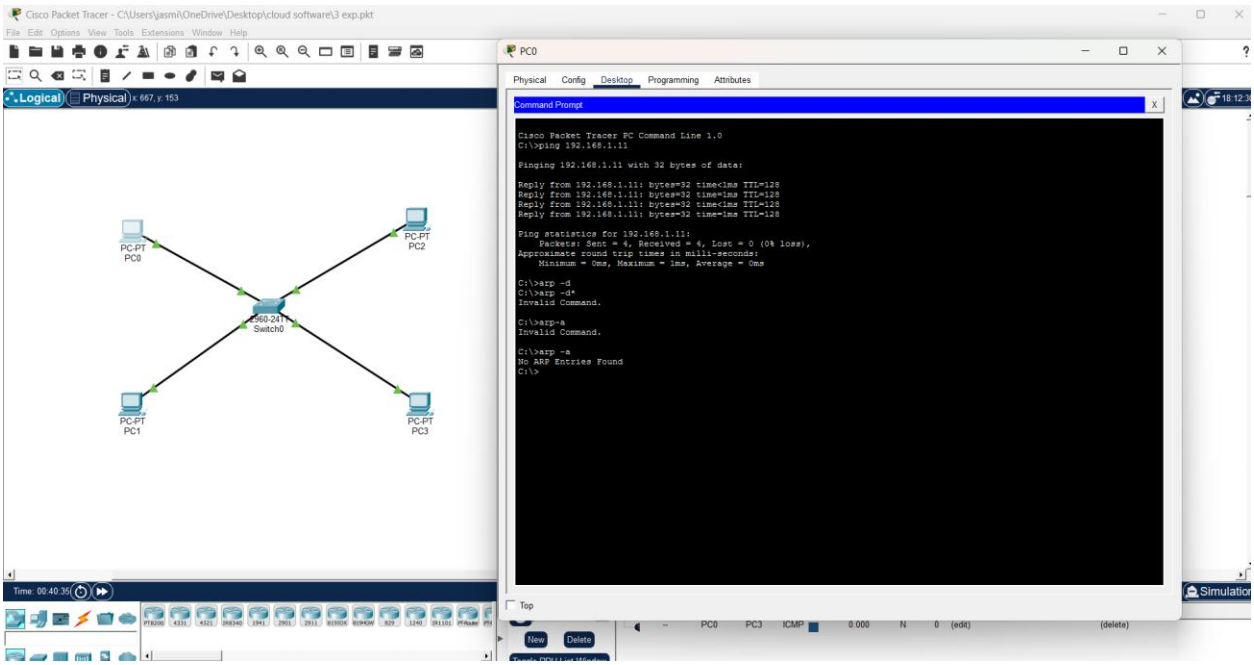


Figure 3: Wireshark capture showing ARP Request and ARP Reply frames

Command 1 — Clearing the ARP Cache

Command 1: Clearing the ARP cache on PC0




```
C:\> arp -d *
```

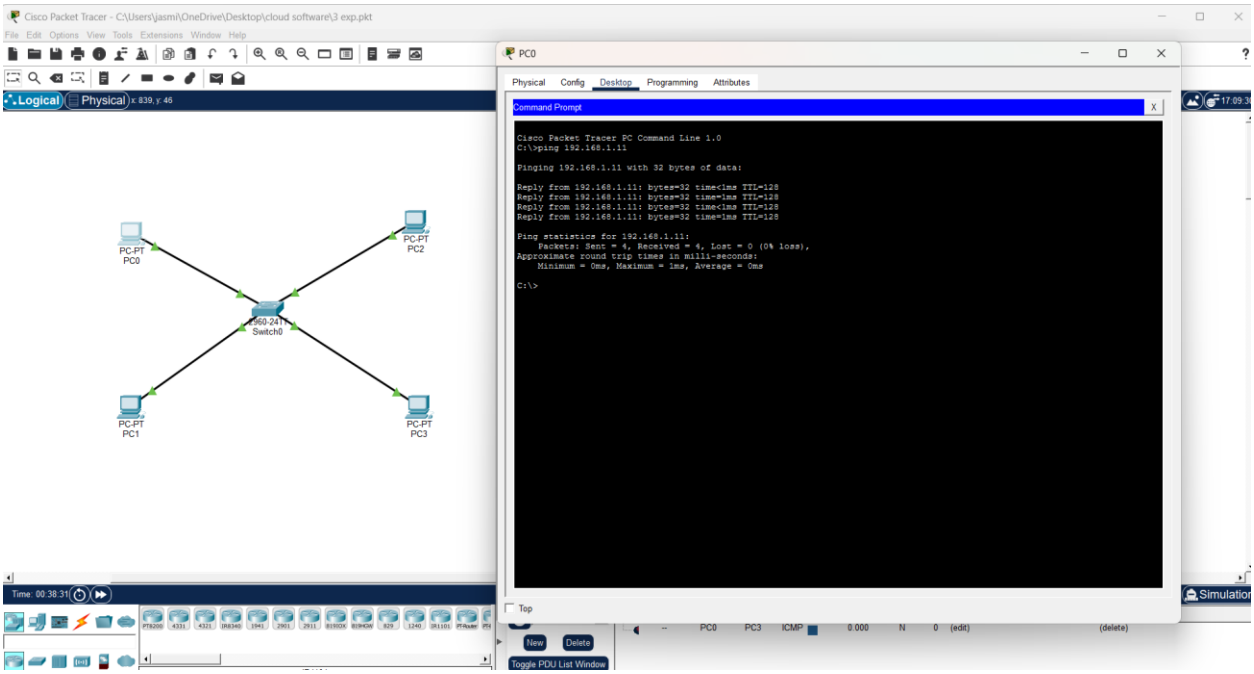
Short Result

```
C:\>arp-a
No ARP Entries Found
```

Explanation / Discussion

The arp -d * command flushes every dynamically learned entry from PC0's ARP cache table. By default, an operating system caches IP-to-MAC mappings for a short period (commonly 120 seconds to a few minutes) so that repeated communication with the same host does not require a fresh ARP exchange every time. Deliberately clearing this cache is essential for this experiment because it forces PC0 to treat 192.168.1.2 as a completely unknown destination at Layer 2, guaranteeing that the very next communication attempt will trigger a visible ARP Request/Reply exchange rather than silently reusing a stale entry. Running arp -a immediately afterward confirms the flush was successful — the empty table ("No ARP Entries Found") is the baseline condition against which the rest of the experiment is measured. Without this step, it would be impossible to reliably observe or capture the ARP broadcast in Wireshark, since a populated cache would let the ping proceed straight to the ICMP exchange.

Command 2 — Initiating Ping



Command 2: Initiating ping from PC0 to PC1

```
C:\> ping 192.168.1.2
```

Short Result

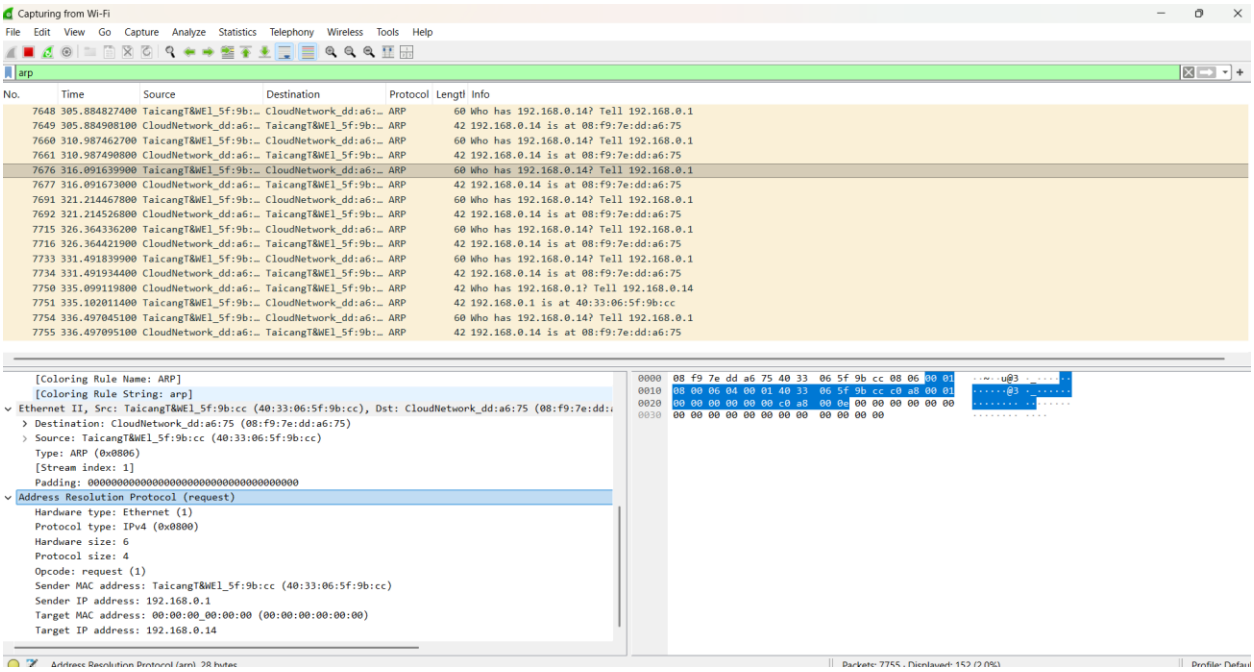
```
Pinging 192.168.1.2 with 32 bytes of data:
Reply from 192.168.1.2: bytes=32 time<1ms TTL=128
Reply from 192.168.1.2: bytes=32 time<1ms TTL=128
Reply from 192.168.1.2: bytes=32 time<1ms TTL=128
Reply from 192.168.1.2: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.1.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
```

Explanation / Discussion

The ping utility operates at the Network layer using ICMP Echo Request/Reply messages, but every ICMP packet must still be wrapped inside an Ethernet II frame before it can physically leave PC0's network interface card. An Ethernet frame header requires a destination MAC address, and since PC0's ARP cache was just cleared, this address is unknown. Consequently, the operating system's network stack automatically pauses the ICMP transmission and internally invokes ARP first, before any ICMP data is placed on the wire. This is why, in Simulation Mode, the very first packet visible is an ARP Request rather than an ICMP packet — the ping command's logical instruction to “send an echo request” cannot be completed until the physical addressing problem is solved. Once ARP resolves the mapping, PC0 immediately encapsulates and transmits four ICMP Echo Requests, and PC1 replies to each, producing the 0% packet loss result shown above. This demonstrates the layered nature of networking: a Layer 3 command (ping) transparently depends on a Layer 2/3 boundary protocol (ARP) to succeed.

Command 3 — Analyzing the ARP Request Packet



Command 3: Inspecting the ARP Request packet fields

Short Result (from PDU / Wireshark inspection)

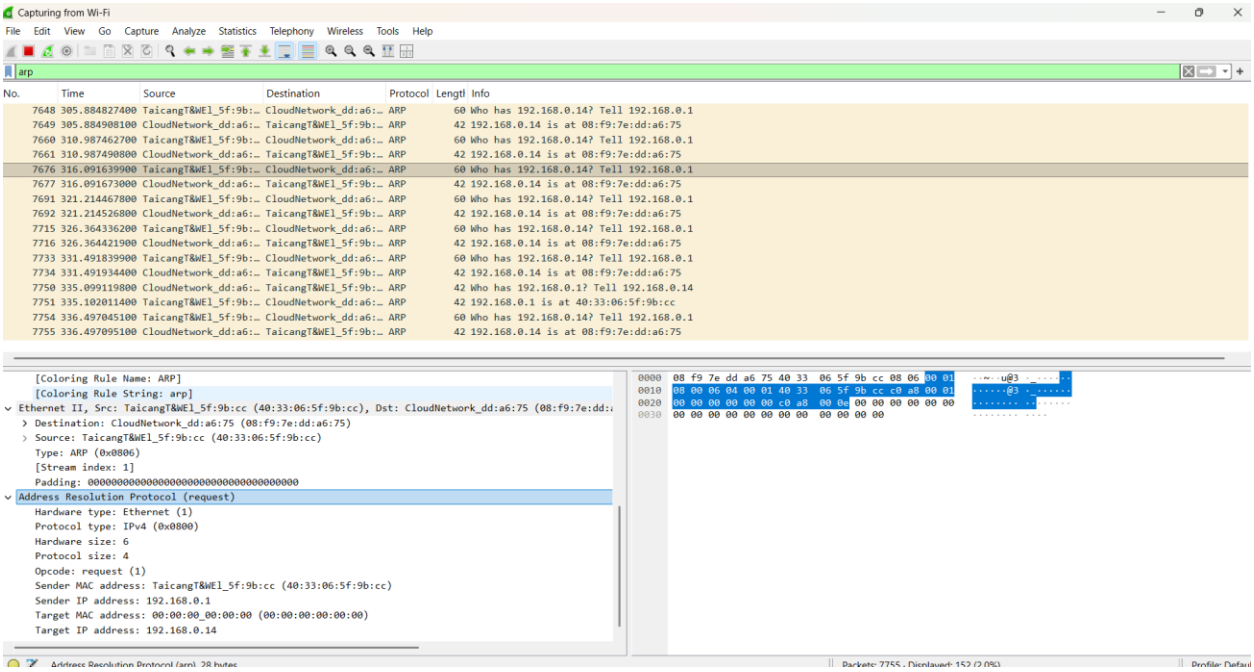
Frame: Ethernet II, Broadcast
Source MAC: 0060.2F3A.1B2C (PC0)
Destination MAC: FFFF.FFFF.FFFF (Broadcast)
Protocol: ARP
Opcode: 1 (Request)
Sender MAC: 0060.2F3A.1B2C
Sender IP: 192.168.1.1
Target MAC: 0000.0000.0000 (Unknown)
Target IP: 192.168.1.2

Explanation / Discussion

The ARP Request frame is addressed to the Ethernet broadcast address FF:FF:FF:FF:FF:FF because PC0 has no way of knowing, in advance, which physical port or host on the LAN owns the IP address 192.168.1.2. Rather than guessing, ARP relies on a one-to-many broadcast so that every device on the local segment can inspect the request. Structurally, the packet's Sender fields (MAC 0060.2F3A.1B2C, IP 192.168.1.1) identify PC0 as the originator, while the Target IP field (192.168.1.2) specifies who must respond; the Target MAC field is deliberately left as 00:00:00:00:00:00 because that is precisely the unknown value being requested. At the switch, this broadcast frame is flooded out of every port except the one it arrived on (standard Layer 2 flooding behavior for broadcast/unknown-destination frames), which is why PC2 also receives a copy even though the request is not meant for it. PC2 compares the Target IP field against its own configured IP (192.168.1.3), finds no match, and silently discards the frame — only PC1 will continue

processing it. This step illustrates ARP's fundamental request mechanism: a controlled broadcast used sparingly and only when a mapping is genuinely unknown.

Command 4 — Analyzing the ARP Reply Packet



Command 4: Inspecting the ARP Reply packet fields

Short Result (from PDU / Wireshark inspection)

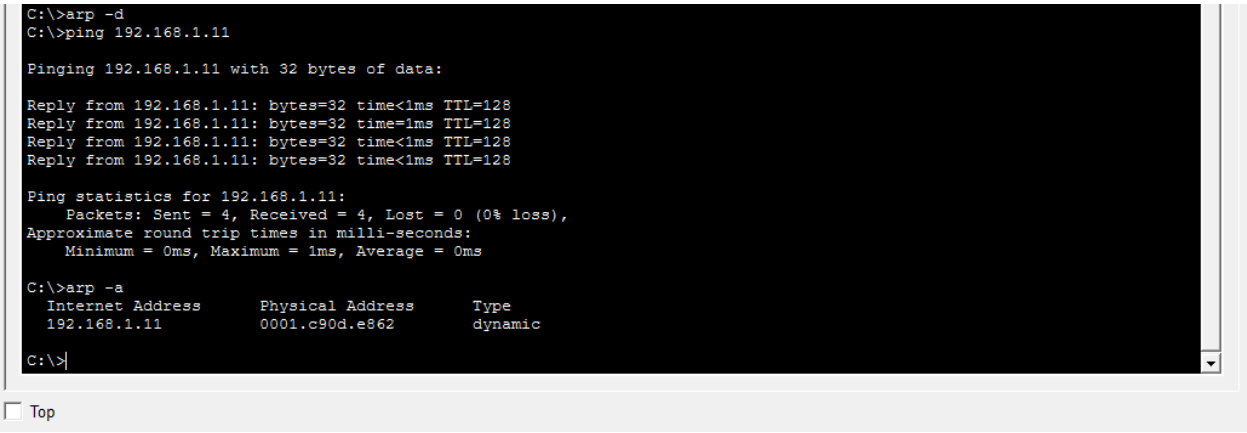
Frame: Ethernet II, Unicast
Source MAC: 0060.2F5B.4D6E (PC1)
Destination MAC: 0060.2F3A.1B2C (PC0)
Protocol: ARP
Opcode: 2 (Reply)
Sender MAC: 0060.2F5B.4D6E
Sender IP: 192.168.1.2
Target MAC: 0060.2F3A.1B2C
Target IP: 192.168.1.1

Explanation / Discussion

Unlike the request, the ARP Reply is never broadcast — it is sent as a direct, unicast Ethernet frame addressed specifically to PC0's MAC address (0060.2F3A.1B2C), because PC1 already learned that address from the Sender MAC field of the incoming request. The Opcode value changes from 1 (Request) to 2 (Reply), signalling to the receiving stack that this frame answers a prior query rather than initiating one. PC1 now populates the Sender fields with its own identity (MAC 0060.2F5B.4D6E, IP 192.168.1.2) and copies PC0's information into the Target fields, effectively reversing the roles from the request packet. This targeted, one-to-one delivery is more efficient than a second broadcast would be, since only the original requester needs the information.

When this frame reaches PC0's network interface, the operating system extracts the Sender MAC/IP pair and immediately writes it into the local ARP cache as a new dynamic entry, which is what finally unblocks the pending ICMP Echo Requests queued behind the ping command.

Command 5 — Verifying the Updated ARP Cache



Command 5: Verifying the updated ARP cache on PC0

```
C:\> arp -a
```

Short Result

```
Interface: 192.168.1.1 --- 0x1
  Internet Address      Physical Address      Type
  192.168.1.2           0060.2f5b.4d6e        dynamic
```

Explanation / Discussion

This final verification step confirms that the ARP transaction actually succeeded and persisted in memory, closing the loop that began with Command 1's deliberate cache flush. The single dynamic entry mapping 192.168.1.2 to 0060.2f5b.4d6e is exactly the information carried in the ARP Reply's Sender fields, proving that PC0 correctly parsed and stored the response. The entry is marked “dynamic” (as opposed to “static”) because it was learned automatically through the ARP protocol and is therefore subject to an aging timer; if no traffic is exchanged with PC1 for the cache's timeout duration, the entry will expire and a fresh ARP Request/Reply cycle will be required the next time PC0 needs to reach 192.168.1.2. Practically, this caching behavior is what makes subsequent pings, file transfers, or any further communication with PC1 faster — PC0 can now build Ethernet frames for PC1 immediately, without repeating the broadcast discovery process, until the entry is cleared or expires.

Overall Result and Discussion

The experiment successfully demonstrated the complete ARP resolution process within a switched LAN, from an empty cache through to a fully populated one, and ties directly back to the five commands executed above. Initially, with PC0's ARP cache deliberately emptied (Command 1),

the host had no record of any IP-to-MAC mapping on the network. When the ping command was issued (Command 2), the operating system could not immediately transmit an ICMP Echo Request because it lacked the destination's physical address — this is the critical dependency that the whole experiment is designed to expose: Layer 3 communication cannot occur without Layer 2 addressing information, and ARP is the mechanism that bridges this gap on demand.

Analysis of the captured frames (Commands 3 and 4) showed the two halves of the ARP transaction behaving exactly as the protocol specifies:

- PC0 broadcast an ARP Request (Opcode 1) asking "Who has 192.168.1.2? Tell 192.168.1.1," addressed to the Ethernet broadcast MAC FF:FF:FF:FF:FF:FF, which the switch flooded out of every port except the source port.
- Every host on the segment, including PC2, received and inspected the frame, but only PC1 — whose IP matched the Target IP field — continued processing it; PC2 discarded it silently.
- PC1 replied with a unicast ARP Reply (Opcode 2) addressed directly to PC0's MAC address, embedding its own MAC/IP pair in the Sender fields so PC0 could learn it in a single exchange.
- PC0 immediately cached this mapping (Command 5) and released the four queued ICMP Echo Requests, all of which received replies with 0% packet loss (Command 2 result).

This sequence confirms several important properties of ARP as a protocol: it operates strictly at the boundary between Layer 3 (IP) and Layer 2 (Ethernet/MAC); it is invoked reactively, only when the required mapping is absent from the local cache, rather than continuously; it uses a broadcast-then-unicast pattern that balances the need to reach an unknown host against the inefficiency of broadcasting the reply as well; and it is stateful, since the learned mapping is cached and reused for subsequent frames rather than being re-requested for every packet. The experiment also highlighted the practical role of the switch, which does not participate in the ARP resolution logic itself but is responsible for correctly flooding the broadcast frame and later forwarding the unicast reply to the correct port based on its MAC address table, ensuring the entire exchange completes within a switched environment rather than requiring a hub-style shared medium.

Conclusion

The Address Resolution Protocol (ARP) is essential for enabling communication in an IP-based Ethernet LAN, as it bridges the gap between logical addressing (IP) used by the Network layer and physical addressing (MAC) used by the Data Link layer. By clearing the ARP cache and observing a fresh ping exchange, it was demonstrated that a host must first discover a destination's MAC address via a broadcast ARP Request before any unicast data (such as ICMP packets) can be transmitted. The corresponding ARP Reply, sent as a unicast frame, completes this resolution and allows the ARP cache to be populated for efficient future communication. This experiment validates the fundamental role of ARP in ensuring seamless host-to-host communication within a local area network.